

---

# DIGITAL ASSESSMENT – 5

---

S. Yuvaresh | 24BCS0195 | Full Stack Application Development

1. Create Mark JSON data file which contains 10 documents with fields such as regno, name of the student and marks of 3 courses. Process the Mark.json using any webserver and perform the following.

a. Find the no of students those who are secured  $\geq 50$  in all courses and display the details in tabular format.

b. Display the slow learner in header1 tag with style of your choice.

**Program:**

**Mark.json:**

```
[
  {
    "regno": "24BCS001",
    "name": "Arun",
    "course1": 78,
    "course2": 85,
    "course3": 72
  },
  {
    "regno": "24BCS002",
    "name": "Bala",
    "course1": 45,
    "course2": 56,
    "course3": 60
  },
  {
    "regno": "24BCS003",
    "name": "Charan",
    "course1": 91,
    "course2": 88,
    "course3": 95
  },
  {
    "regno": "24BCS004",
    "name": "Deepa",
    "course1": 67,
    "course2": 74,
    "course3": 81
  },
  {
    "regno": "24BCS005",
    "name": "Ezhil",
    "course1": 32,
    "course2": 45,
    "course3": 39
  },
  {
    "regno": "24BCS006",
```

```

    "name": "Farhan",
    "course1": 55,
    "course2": 62,
    "course3": 58
  },
  {
    "regno": "24BCS007",
    "name": "Gokul",
    "course1": 49,
    "course2": 51,
    "course3": 47
  },
  {
    "regno": "24BCS008",
    "name": "Harini",
    "course1": 83,
    "course2": 79,
    "course3": 86
  },
  {
    "regno": "24BCS009",
    "name": "Ishaan",
    "course1": 61,
    "course2": 68,
    "course3": 73
  },
  {
    "regno": "24BCS010",
    "name": "Janani",
    "course1": 38,
    "course2": 42,
    "course3": 35
  }
]

```

### markprocess.html:

```

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mark Processing</title>
  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  <!-- jQuery -->
  <script src="min_js_files/jquery-3.6.3.min.js"></script>
</head>

<body class="bg-light">

```

```

<h3>S.Yuvaresh</h3>
<h3>24BCS0195</h3>
<div class="container mt-5">
  <div class="card shadow">
    <div class="card-header bg-primary text-white text-center">
      <h2>Student Mark Processing</h2>
    </div>
    <div class="card-body">
      <!-- Question 1(b) -->
      <h1 id="slowLearner" class="text-center alert alert-danger mb-4">
      </h1>
      <!-- Question 1(a) -->
      <h3 class="text-primary">
        Students Secured >= 50 in All Courses
      </h3>
      <div class="alert alert-success">
        <strong>
          Number of Students:
        </strong>
        <span id="studentCount"></span>
      </div>
      <div class="table-responsive">
        <table class="table table-bordered table-striped table-hover">
          <thead class="table-dark">
            <tr>
              <th>Register Number</th>
              <th>Name</th>
              <th>Course 1</th>
              <th>Course 2</th>
              <th>Course 3</th>
            </tr>
          </thead>
          <tbody id="studentTable">
          </tbody>
        </table>
      </div>
    </div>
  </div>
</div>
<script>
  $(document).ready(function () {
    $.getJSON("Mark.json", function (data) {
      let count = 0;
      let tableData = "";
      let slowLearner = data[0];
      let lowestAverage =
        (data[0].course1 +
          data[0].course2 +
          data[0].course3) / 3;
      $.each(data, function (index, student) {
        let average =
          (student.course1 +

```

```

        student.course2 +
        student.course3) / 3;
    /*
    * Question 1(a)
    * Find students who scored >= 50
    * in all three courses.
    */
    if (
        student.course1 >= 50 &&
        student.course2 >= 50 &&
        student.course3 >= 50
    ) {
        count++;
        tableData += `
<tr>
<td>${student.regno}</td>
<td>${student.name}</td>
<td>${student.course1}</td>
<td>${student.course2}</td>
<td>${student.course3}</td>
</tr>
`;
    }
    /*
    * Question 1(b)
    * Find the student with the
    * lowest average mark.
    */
    if (average < lowestAverage) {
        lowestAverage = average;
        slowLearner = student;
    }
});
// Display count
$("#studentCount").text(count);
// Display table
$("#studentTable").html(tableData);
// Display slow learner in H1
$("#slowLearner").html(
    "Slow Learner: " +
    slowLearner.name +
    " | Average Mark: " +
    lowestAverage.toFixed(2)
);
});
});
</script>
</body>

</html>

```

**Output:**

VIT Vellore - VTOP x 24BCS0195\_VL2020270 x Mark Processing x Index of /exam x +

127.0.0.1:5500/markprocess.html

S.Yuvaresh  
24BCS0195

Student Mark Processing

Slow Learner: Janani | Average Mark: 38.33

Students Secured >= 50 in All Courses

Number of Students: 6

| Register Number | Name   | Course 1 | Course 2 | Course 3 |
|-----------------|--------|----------|----------|----------|
| 24BCS001        | Arun   | 78       | 85       | 72       |
| 24BCS003        | Charan | 91       | 88       | 95       |
| 24BCS004        | Deepa  | 67       | 74       | 81       |
| 24BCS006        | Farhan | 55       | 62       | 58       |
| 24BCS008        | Harini | 83       | 79       | 86       |
| 24BCS009        | Ishaan | 61       | 68       | 73       |

2. Create Stock JSON data file which contains 10 documents with fields such as item no, name of the item, quantity and price of the item Process the Stock.json using any webserver and perform the following

- Find the no of products whose stock are equal to 50 and display the stock details in ordered list
- Find the no of out of stock products.

### **Program:**

### **Stock.json:**

```
[
  {
    "itemno": 101,
    "name": "Laptop",
    "quantity": 50,
    "price": 55000
  },
  {
    "itemno": 102,
    "name": "Mouse",
    "quantity": 25,
    "price": 800
  },
  {
    "itemno": 103,
    "name": "Keyboard",
    "quantity": 50,
    "price": 1500
  }
]
```

```

    },
    {
        "itemno": 104,
        "name": "Monitor",
        "quantity": 0,
        "price": 12000
    },
    {
        "itemno": 105,
        "name": "Printer",
        "quantity": 15,
        "price": 8500
    },
    {
        "itemno": 106,
        "name": "Scanner",
        "quantity": 0,
        "price": 7000
    },
    {
        "itemno": 107,
        "name": "Webcam",
        "quantity": 50,
        "price": 2500
    },
    {
        "itemno": 108,
        "name": "Speaker",
        "quantity": 30,
        "price": 1800
    },
    {
        "itemno": 109,
        "name": "Headphone",
        "quantity": 10,
        "price": 2200
    },
    {
        "itemno": 110,
        "name": "USB Drive",
        "quantity": 50,
        "price": 700
    }
]

```

### **stockprocess.html:**

```

<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">

```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Stock Processing</title>
<!-- Bootstrap CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
<!-- jQuery -->
<script src="min_js_files/jquery-3.6.3.min.js"></script>
</head>

<body class="bg-light">
  <div class="container mt-5">
    <div class="card shadow">
      <div class="card-header bg-success text-white text-center">
        <h2>Stock Processing</h2>
      </div>
      <div class="card-body">
        <!-- Question 2(c) -->
        <h3 class="text-success">
          Products Having Stock Equal to 50
        </h3>
        <div class="alert alert-primary">
          <strong>
            Number of Products:
          </strong>
          <span id="stockCount"></span>
        </div>
        <!-- Ordered List -->
        <ol id="stockList" class="list-group list-group-numbered mb-5">
        </ol>
        <!-- Question 2(d) -->
        <h3 class="text-danger">
          Out of Stock Products
        </h3>
        <div class="alert alert-danger">
          <strong>
            Number of Out of Stock Products:
          </strong>
          <span id="outOfStockCount"></span>
        </div>
      </div>
    </div>
  </div>
</div>
<script>
  $(document).ready(function () {
    $.getJSON("Stock.json", function (data) {
      let stockCount = 0;
      let outOfStockCount = 0;
      let stockList = "";
      $.each(data, function (index, item) {
        /*
         * Question 2(c)
         * Find products whose quantity is 50.

```



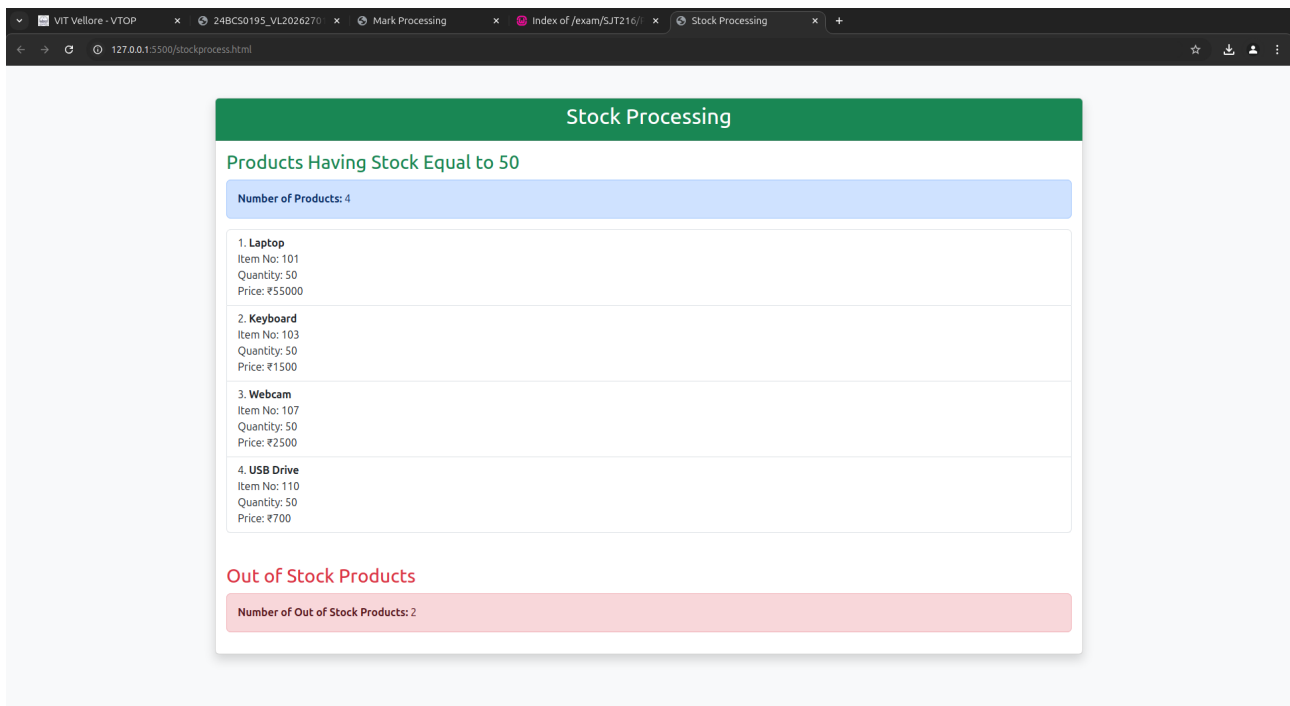
```

        */
        if (item.quantity === 50) {
            stockCount++;
            stockList += `
<li class="list-group-item">
<strong>
${item.name}
</strong>
<br>
Item No:
${item.itemno}
<br>
Quantity:
${item.quantity}
<br>
Price:
₹${item.price}
</li>
`;
        }
    /*
    * Question 2(d)
    * Find out-of-stock products.
    */
    if (item.quantity === 0) {
        outOfStockCount++;
    }
    });
    // Display number of products with stock 50
    $("#stockCount").text(stockCount);
    // Display products in ordered list
    $("#stockList").html(stockList);
    // Display number of out-of-stock products
    $("#outOfStockCount").text(outOfStockCount);
    });
    });
</script>
</body>

</html>

```

**Output:**



### 3. MongoDB exercises

Insert the following data into persons collection:

```
[
  {
    "name": "Sue Ramsey",
    "age": 35,
    "country": "Algeria"
  },
  {
    "name": "Leah McLaughlin",
    "age": 29,
    "country": "Japan"
  },
  {
    "name": "Luke Chandler",
    "age": 20,
    "country": "Argentina"
  },
  {
    "name": "Douglas Harrington",
    "age": 29,
    "country": "United Kingdom"
  },
  {
    "name": "Esther Lamb",
    "age": 30,
    "country": "Italy"
  }
]
```

```
},
{
  "name": "Viola Gomez",
  "age": 34,
  "country": "United States"
},
{
  "name": "Beatrice Webster",
  "age": 17,
  "country": "India"
},
{
  "name": "Steve Webster",
  "age": 33,
  "country": "Japan"
},
{
  "name": "Mollie Estrada",
  "age": 24,
  "country": "Russia"
},
{
  "name": "Susie Fisher",
  "age": 28,
  "country": "Canada"
},
{
  "name": "Loretta Reynolds",
  "age": 27,
  "country": "United States"
},
{
  "name": "Lou Campbell",
  "age": 34,
  "country": "France"
},
{
  "name": "Marion Henry",
  "age": 33,
  "country": "India"
},
{
  "name": "Daniel Thomas",
  "age": 20,
  "country": "Italy"
},
{
  "name": "Sarah Palmer",
  "age": 18,
  "country": "India"
},
{
```

```

    "name": "Emilie Stevenson",
    "age": 35,
    "country": "Japan"
  },
  {
    "name": "Manuel Knight",
    "age": 33,
    "country": "United Kingdom"
  },
  {
    "name": "Nathan Howell",
    "age": 29,
    "country": "Germany"
  },
  {
    "name": "Brent Thornton",
    "age": 21,
    "country": "United States"
  },
]

```

### **Program:**

```

db.persons.insertMany([
  {
    name: "Sue Ramsey",
    age: 35,
    country: "Algeria"
  },
  {
    name: "Leah McLaughlin",
    age: 29,
    country: "Japan"
  },
  {
    name: "Luke Chandler",
    age: 20,
    country: "Argentina"
  },
  {
    name: "Douglas Harrington",
    age: 29,
    country: "United Kingdom"
  },
  {
    name: "Esther Lamb",
    age: 30,
    country: "Italy"
  },
])

```

```
{
  name: "Viola Gomez",
  age: 34,
  country: "United States"
},
{
  name: "Beatrice Webster",
  age: 17,
  country: "India"
},
{
  name: "Steve Webster",
  age: 33,
  country: "Japan"
},
{
  name: "Mollie Estrada",
  age: 24,
  country: "Russia"
},
{
  name: "Susie Fisher",
  age: 28,
  country: "Canada"
},
{
  name: "Loretta Reynolds",
  age: 27,
  country: "United States"
},
{
  name: "Lou Campbell",
  age: 34,
  country: "France"
},
{
  name: "Marion Henry",
  age: 33,
  country: "India"
},
{
  name: "Daniel Thomas",
  age: 20,
  country: "Italy"
},
{
  name: "Sarah Palmer",
  age: 18,
  country: "India"
},
{
  name: "Emilie Stevenson",
```

```

    age: 35,
    country: "Japan"
  },
  {
    name: "Manuel Knight",
    age: 33,
    country: "United Kingdom"
  },
  {
    name: "Nathan Howell",
    age: 29,
    country: "Germany"
  },
  {
    name: "Brent Thornton",
    age: 21,
    country: "United States"
  }
]);

```

## Output:

```

24BC50195> db.persons.insertMany([
  {name: "Sue Ramsey",age: 35,country: "Algeria"},{name: "Leah McLaughlin",age: 29,country: "Japan"},{name: "Luke Chandler",age: 20,country: "Argentina"},{name: "Douglas Harrington",age: 29,country: "United Kingdom"},{name: "Esther Lamb",age: 30,country: "Italy"},{name: "Viola Gomez",age: 34,country: "United States"},{name: "Beatrice Webster",age: 17,country: "India"},{name: "Steve Webster",age: 33,country: "Japan"},{name: "Mollie Estrada",age: 24,country: "Russia"},{name: "Susie Fisher",age: 28,country: "Canada"},{name: "Loretta Reynolds",age: 27,country: "United States"},{name: "Lou Campbell",age: 34,country: "France"},{name: "Marlon Henry",age: 33,country: "India"},{name: "Daniel Thomas",age: 20,country: "Italy"},{name: "Sarah Palmer",age: 18,country: "India"},{name: "Emilie Stevenson",age: 35,country: "Japan"},{name: "Manuel Knight",age: 33,country: "United Kingdom"},{name: "Nathan Howell",age: 29,country: "Germany"},{name: "Brent Thornton",age: 21,country: "United States"}]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a9f7f2643e672b0cffe6911'),
    '1': ObjectId('6a9f7f2643e672b0cffe6912'),
    '2': ObjectId('6a9f7f2643e672b0cffe6913'),
    '3': ObjectId('6a9f7f2643e672b0cffe6914'),
    '4': ObjectId('6a9f7f2643e672b0cffe6915'),
    '5': ObjectId('6a9f7f2643e672b0cffe6916'),
    '6': ObjectId('6a9f7f2643e672b0cffe6917'),
    '7': ObjectId('6a9f7f2643e672b0cffe6918'),
    '8': ObjectId('6a9f7f2643e672b0cffe6919'),
    '9': ObjectId('6a9f7f2643e672b0cffe691a'),
    '10': ObjectId('6a9f7f2643e672b0cffe691b'),
    '11': ObjectId('6a9f7f2643e672b0cffe691c'),
    '12': ObjectId('6a9f7f2643e672b0cffe691d'),
    '13': ObjectId('6a9f7f2643e672b0cffe691e'),
    '14': ObjectId('6a9f7f2643e672b0cffe691f'),
    '15': ObjectId('6a9f7f2643e672b0cffe6920'),
    '16': ObjectId('6a9f7f2643e672b0cffe6921'),
    '17': ObjectId('6a9f7f2643e672b0cffe6922'),
    '18': ObjectId('6a9f7f2643e672b0cffe6923')
  }
}
24BC50195>

```

1. Find all persons between the age of 20 to 30.

## Program:

```
db.persons.find({age: {$gte: 20,$lte: 30}});
```

## Output:

```
mongosh mongodbg://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
24BC50195> db.persons.find({age: {$gte: 20,$lte: 30}});
[
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6912'),
    name: 'Leah McLaughlin',
    age: 29,
    country: 'Japan'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6913'),
    name: 'Luke Chandler',
    age: 20,
    country: 'Argentina'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6914'),
    name: 'Douglas Harrington',
    age: 29,
    country: 'United Kingdom'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6915'),
    name: 'Esther Lamb',
    age: 30,
    country: 'Italy'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6919'),
    name: 'Mollie Estrada',
    age: 24,
    country: 'Russia'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe691a'),
    name: 'Susie Fisher',
    age: 28,
    country: 'Canada'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe691b'),
    name: 'Loretta Reynolds',
    age: 27,
    country: 'United States'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe691c'),
    name: 'Daniel Thomas',
    age: 20,
    country: 'Italy'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6922'),
    name: 'Nathan Howell',
    age: 29,
    country: 'Germany'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6923'),
    name: 'Brent Thornton',
    age: 21,
    country: 'United States'
  }
]
24BC50195>
```

2. Find all persons except those whose name isn't in ["Nathan Howell", "Marion Henry", "Emilie Stevenson"]

### Program:

```
db.persons.find({name: {$nin: ["Nathan Howell","Marion Henry","Emilie Stevenson"]}});
```

### Output:

```

24BC50195> db.persons.find((name: {$in: ["Nathan Howell", "Marion Henry", "Entile Stevenson"]}));
[
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6911'),
    name: 'Sue Ramsey',
    age: 35,
    country: 'Algeria'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6912'),
    name: 'Leah McLaughlin',
    age: 29,
    country: 'Japan'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6913'),
    name: 'Luke Chandler',
    age: 20,
    country: 'Argentina'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6914'),
    name: 'Douglas Harrington',
    age: 29,
    country: 'United Kingdom'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6915'),
    name: 'Esther Lamb',
    age: 30,
    country: 'Italy'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6916'),
    name: 'Viola Gomez',
    age: 34,
    country: 'United States'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6917'),
    name: 'Beatrice Webster',
    age: 17,
    country: 'India'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6918'),
    name: 'Steve Webster',
    age: 33,
    country: 'Japan'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6919'),
    name: 'Mollie Estrada',
    age: 24,
    country: 'Russia'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe691a'),
    name: 'Susie Fisher',
    age: 28,
    country: 'Canada'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe691b'),
    name: 'Loretta Reynolds',
    age: 27,
    country: 'United States'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe691c'),
    name: 'Lou Campbell',
    age: 34,
    country: 'France'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe691e'),
    name: 'Daniel Thomas',
    age: 28,
    country: 'Italy'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe691f'),
    name: 'Sarah Palmer',
    age: 18,
    country: 'India'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6920'),
    name: 'Manuel Knight',
    age: 33,
    country: 'United Kingdom'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6923'),
    name: 'Brent Thornton',
    age: 21,
    country: 'United States'
  }
]

```

```

country: 'India'
},
{
  _id: ObjectId('6a9f7f2643e672b0cffe6918'),
  name: 'Steve Webster',
  age: 33,
  country: 'Japan'
},
{
  _id: ObjectId('6a9f7f2643e672b0cffe6919'),
  name: 'Mollie Estrada',
  age: 24,
  country: 'Russia'
},
{
  _id: ObjectId('6a9f7f2643e672b0cffe691a'),
  name: 'Susie Fisher',
  age: 28,
  country: 'Canada'
},
{
  _id: ObjectId('6a9f7f2643e672b0cffe691b'),
  name: 'Loretta Reynolds',
  age: 27,
  country: 'United States'
},
{
  _id: ObjectId('6a9f7f2643e672b0cffe691c'),
  name: 'Lou Campbell',
  age: 34,
  country: 'France'
},
{
  _id: ObjectId('6a9f7f2643e672b0cffe691e'),
  name: 'Daniel Thomas',
  age: 28,
  country: 'Italy'
},
{
  _id: ObjectId('6a9f7f2643e672b0cffe691f'),
  name: 'Sarah Palmer',
  age: 18,
  country: 'India'
},
{
  _id: ObjectId('6a9f7f2643e672b0cffe6920'),
  name: 'Manuel Knight',
  age: 33,
  country: 'United Kingdom'
},
{
  _id: ObjectId('6a9f7f2643e672b0cffe6923'),
  name: 'Brent Thornton',
  age: 21,
  country: 'United States'
}
]
24BC50195>

```

```

country: 'India'
},
{
  _id: ObjectId('6a9f7f2643e672b0cffe6921'),
  name: 'Manuel Knight',
  age: 33,
  country: 'United Kingdom'
},
{
  _id: ObjectId('6a9f7f2643e672b0cffe6923'),
  name: 'Brent Thornton',
  age: 21,
  country: 'United States'
}
]
24BC50195>

```

3. Find all persons in one of these countries ["Canada", "United States", "France"]

### Program:

```
db.persons.find({country: {$in: ["Canada", "United States", "France"]}});
```



### Output:

```
24BCS0195> db.persons.find({country: {$in: ["Canada","United States","France"]}});
[
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6916'),
    name: 'Viola Gomez',
    age: 34,
    country: 'United States'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe691a'),
    name: 'Susie Fisher',
    age: 28,
    country: 'Canada'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe691b'),
    name: 'Loretta Reynolds',
    age: 27,
    country: 'United States'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe691c'),
    name: 'Lou Campbell',
    age: 34,
    country: 'France'
  },
  {
    _id: ObjectId('6a9f7f2643e672b0cffe6923'),
    name: 'Brent Thornton',
    age: 21,
    country: 'United States'
  }
]
24BCS0195> █
```

4. Set every person's gender to male

### Program:

```
db.persons.updateMany({},{$set: {gender: "male"}});
```

### Output:

```
24BCS0195> db.persons.updateMany({},{$set: {gender: "male"}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 19,
  modifiedCount: 19,
  upsertedCount: 0
}
```

5. Remove the gender field from every document

**Program:**

```
db.persons.updateMany({},{$unset: {gender: ""}});
```

**Output:**

```
24BCS0195> db.persons.updateMany({},{$unset: {gender: ""}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 19,
  modifiedCount: 19,
  upsertedCount: 0
}
```

6. Rename name to fullName

**Program:**

```
db.persons.updateMany({},{$rename: {"name": "fullName"}});
```

**Output:**

```
24BCS0195> db.persons.updateMany({},{$rename: {"name": "fullName"}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 19,
  modifiedCount: 19,
  upsertedCount: 0
}
```

7. Rename fullName back to name

**Program:**

```
db.persons.updateMany({},{$rename: {"fullName": "name"}});
```

**Output:**

```
24BCS0195> db.persons.updateMany({},{$rename: {"fullName": "name"}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 19,
  modifiedCount: 19,
  upsertedCount: 0
}
```

8. Find the count of documents

**Program:**

```
db.persons.countDocuments();
```

**Output:**

```
24BCS0195> db.persons.countDocuments();
19
```

9. How many people live in each country?

**Program:**

```
db.persons.aggregate([{$group: {_id: "$country",count: {$sum: 1}}},{ $project: {_id: 0,country: "$_id",count: 1}}]);
```

**Output:**

```
24BCS0195> db.persons.aggregate([{$group: {_id: "$country",count: {$sum: 1}}},{ $project: {_id: 0,country: "$_id",count: 1}}]);
[
  { count: 3, country: 'India' },
  { count: 1, country: 'Argentina' },
  { count: 1, country: 'Canada' },
  { count: 2, country: 'Italy' },
  { count: 1, country: 'Germany' },
  { count: 1, country: 'Russia' },
  { count: 1, country: 'France' },
  { count: 3, country: 'Japan' },
  { count: 2, country: 'United Kingdom' },
  { count: 1, country: 'Algeria' },
  { count: 3, country: 'United States' }
]
```

10. How many people live in India?

**Program:**

```
db.persons.countDocuments({country: "India"});
```

**Output:**

```
24BCS0195> db.persons.countDocuments({country: "India"});
3
```

11. Find the average age of each country

### Program:

```
db.persons.aggregate([{$group: {_id: "$country",averageAge: {$avg: "$age"}}},{$project: {_id: 0,country: "$_id",averageAge: 1}}]);
```

### Output:

```
24BCS0195> db.persons.aggregate([{$group: {_id: "$country",averageAge: {$avg: "$age"}}},{$project: {_id: 0,country: "$_id",averageAge: 1}}]);
[
  { averageAge: 31, country: 'United Kingdom' },
  { averageAge: 20, country: 'Argentina' },
  { averageAge: 22.666666666666668, country: 'India' },
  { averageAge: 28, country: 'Canada' },
  { averageAge: 24, country: 'Russia' },
  { averageAge: 34, country: 'France' },
  { averageAge: 25, country: 'Italy' },
  { averageAge: 29, country: 'Germany' },
  { averageAge: 32.333333333333336, country: 'Japan' },
  { averageAge: 27.333333333333332, country: 'United States' },
  { averageAge: 35, country: 'Algeria' }
]
```

12. Find people belonging to each country

### Program:

```
db.persons.aggregate([{$group: {_id: "$country",persons: {$push: "$name"}}},{$project: {_id: 0,country: "$_id",persons: 1}}]);
```

### Output:

```
24BCS0195> db.persons.aggregate([{$group: {_id: "$country",persons: {$push: "$name"}}},{$project: {_id: 0,country: "$_id",persons: 1}}]);
[
  {
    persons: [ 'Douglas Harrington', 'Manuel Knight' ],
    country: 'United Kingdom'
  },
  {
    persons: [ 'Luke Chandler' ],
    country: 'Argentina'
  },
  {
    persons: [ 'Beatrice Webster', 'Marion Henry', 'Sarah Palmer' ],
    country: 'India'
  },
  {
    persons: [ 'Susie Fisher' ],
    country: 'Canada'
  },
  {
    persons: [ 'Mollie Estrada' ],
    country: 'Russia'
  },
  {
    persons: [ 'Lou Campbell' ],
    country: 'France'
  },
  {
    persons: [ 'Esther Lamb', 'Daniel Thomas' ],
    country: 'Italy'
  },
  {
    persons: [ 'Nathan Howell' ],
    country: 'Germany'
  },
  {
    persons: [ 'Leah McLaughlin', 'Steve Webster', 'Emilie Stevenson' ],
    country: 'Japan'
  },
  {
    persons: [ 'Viola Gomez', 'Loretta Reynolds', 'Brent Thornton' ],
    country: 'United States'
  },
  {
    persons: [ 'Sue Ramsey' ],
    country: 'Algeria'
  }
]
```

#### 4. Insert the following data into movies collection

```
{
  title: "Fight Club",
  writer: "Chuck Palahniuk",
  year: 1999,
  actors: [
    "Brad Pitt",
    "Edward Norton"
  ]
},
{
  title: "Pulp Fiction",
  writer: "Quentin Tarantino",
  year: 1994,
  actors: [
    "John Travolta",
    "Uma Thurman"
  ]
},
{
  title: "Inglorious Basterds",
  writer: "Quentin Tarantino",
  year: 2009,
  actors: [
    "Brad Pitt",
    "Diane Kruger",
    "Eli Roth"
  ]
},
{
  title: "The Hobbit: An Unexpected Journey",
  writer: "J.R.R. Tolkein",
  year: 2012,
  franchise: "The Hobbit"
},
{
  title: "The Hobbit: The Desolation of Smaug",
  writer: "J.R.R. Tolkein",
  year: 2013,
  franchise: "The Hobbit"
},
{
  title: "The Hobbit: The Battle of the Five Armies",
  writer: "J.R.R. Tolkein",
  year: 2012,
  franchise: "The Hobbit",
  synopsis: "Bilbo and Company are forced to engage in a war against an array of combatants and keep the Lonely Mountain from falling into the hands of a rising darkness."
},
{
```

```

    title: "Pee Wee Herman's Big Adventure"
  },
  {
    title: "Avatar"
  }
}

```

### Program:

```

db.movies.insertMany([
  {title: "Fight Club",writer: "Chuck Palahniuk",year: 1999,actors: [
    "Brad Pitt",
    "Edward Norton"
  ]
},
  {title: "Pulp Fiction",writer: "Quentin Tarantino",year: 1994,actors: [
    "John Travolta",
    "Uma Thurman"
  ]
},
  {title: "Inglorious Basterds",writer: "Quentin Tarantino",year: 2009,actors: [
    "Brad Pitt",
    "Diane Kruger",
    "Eli Roth"
  ]
},
  {title: "The Hobbit: An Unexpected Journey",writer: "J.R.R. Tolkein",year: 2012,franchise: "The Hobbit"
},
  {title: "The Hobbit: The Desolation of Smaug",writer: "J.R.R. Tolkein",year: 2013,franchise: "The Hobbit"
},
  {title: "The Hobbit: The Battle of the Five Armies",writer: "J.R.R. Tolkein",year: 2012,franchise: "The Hobbit",synopsis: "Bilbo and Company are forced to engage in a war against an array of combatants and keep the Lonely Mountain from falling into the hands of a rising darkness."
},
  {title: "Pee Wee Herman's Big Adventure"
},
  {title: "Avatar"
}
]);

```

### Output:

```

24BC58195> db.movies.insertMany([{"title": "Fight Club",writer: "Chuck Palahniuk",year: 1999,actors: ["Brad Pitt","Edward Norton"]}, {"title": "Pulp Fiction",writer: "Quentin Tarantino",year: 1994,actors: ["John Travolta","Uma Thurman"]}, {"title": "Inglorious Basterds",writer: "Quentin Tarantino",year: 2009,actors: ["Brad Pitt","Diane Kruger","Eli Roth"]}, {"title": "The Hobbit: An Unexpected Journey",writer: "J.R.R. Tolkein",year: 2012,franchise: "The Hobbit"}, {"title": "The Hobbit: The Desolation of Smaug",writer: "J.R.R. Tolkein",year: 2013,franchise: "The Hobbit"}, {"title": "The Hobbit: The Battle of the Five Armies",writer: "J.R.R. Tolkein",year: 2012,franchise: "The Hobbit",synopsis: "Bilbo and Company are forced to engage in a war against an array of combatants and keep the Lonely Mountain from falling into the hands of a rising darkness."}, {"title": "Pee Wee Herman's Big Adventure"}, {"title": "Avatar"}]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a9f84ab43e672b0cffe6924'),
    '1': ObjectId('6a9f84ab43e672b0cffe6925'),
    '2': ObjectId('6a9f84ab43e672b0cffe6926'),
    '3': ObjectId('6a9f84ab43e672b0cffe6927'),
    '4': ObjectId('6a9f84ab43e672b0cffe6928'),
    '5': ObjectId('6a9f84ab43e672b0cffe6929'),
    '6': ObjectId('6a9f84ab43e672b0cffe692a'),
    '7': ObjectId('6a9f84ab43e672b0cffe692b')
  }
}

```

### 1. Retrieve all documents

## Program:

```
db.movies.find().pretty();
```

## Output:

```
24BC58195> db.movies.find().pretty();
[
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6924'),
    title: 'Fight Club',
    writer: 'Chuck Palahniuk',
    year: 1999,
    actors: [ 'Brad Pitt', 'Edward Norton' ]
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6925'),
    title: 'Pulp Fiction',
    writer: 'Quentin Tarantino',
    year: 1994,
    actors: [ 'John Travolta', 'Uma Thurman' ]
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6926'),
    title: 'Inglorious Basterds',
    writer: 'Quentin Tarantino',
    year: 2009,
    actors: [ 'Brad Pitt', 'Diane Kruger', 'Eli Roth' ]
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6927'),
    title: 'The Hobbit: An Unexpected Journey',
    writer: 'J.R.R. Tolkien',
    year: 2012,
    franchise: 'The Hobbit'
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6928'),
    title: 'The Hobbit: The Desolation of Smaug',
    writer: 'J.R.R. Tolkien',
    year: 2013,
    franchise: 'The Hobbit'
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6929'),
    title: 'The Hobbit: The Battle of the Five Armies',
    writer: 'J.R.R. Tolkien',
    year: 2012,
    franchise: 'The Hobbit',
    synopsis: 'Bilbo and Company are forced to engage in a war against an array of combatants and keep the Lonely Mountain from falling into the hands of a rising darkness.'
  },
]
```

```
{
  _id: ObjectId('6a9f84ab43e672b0cffe692a'),
  title: 'Pee Wee Herman's Big Adventure'
},
{ _id: ObjectId('6a9f84ab43e672b0cffe692b'), title: 'Avatar' }
]
24BC58195>
```

## 2. Retrieve all documents with “Quentin Tarantino” Writer

### Program:

```
db.movies.find({writer: "Quentin Tarantino"});
```

### Output:

```
]
24BCS0195> db.movies.find({writer: "Quentin Tarantino"});
[
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6925'),
    title: 'Pulp Fiction',
    writer: 'Quentin Tarantino',
    year: 1994,
    actors: [ 'John Travolta', 'Uma Thurman' ]
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6926'),
    title: 'Inglorious Basterds',
    writer: 'Quentin Tarantino',
    year: 2009,
    actors: [ 'Brad Pitt', 'Diane Kruger', 'Eli Roth' ]
  }
]
24BCS0195> 
```

3. Retrieve all documents whose actors contain “Brad Pitt”

**Program:**

```
db.movies.find({actors: "Brad Pitt"});
```

**Output:**



```

]
24BCS0195> db.movies.find({actors: "Brad Pitt"});
[
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6924'),
    title: 'Fight Club',
    writer: 'Chuck Palahniuk',
    year: 1999,
    actors: [ 'Brad Pitt', 'Edward Norton' ]
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6926'),
    title: 'Inglorious Basterds',
    writer: 'Quentin Tarantino',
    year: 2009,
    actors: [ 'Brad Pitt', 'Diane Kruger', 'Eli Roth' ]
  }
]
24BCS0195> █

```

4. Retrieve all documents with “The Hobbit” franchise

**Program:**

```
db.movies.find({franchise: "The Hobbit"});
```

**Output:**

```

24BCS0195> db.movies.find({franchise: "The Hobbit"});
[
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6927'),
    title: 'The Hobbit: An Unexpected Journey',
    writer: 'J.R.R. Tolkien',
    year: 2012,
    franchise: 'The Hobbit'
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6928'),
    title: 'The Hobbit: The Desolation of Smaug',
    writer: 'J.R.R. Tolkien',
    year: 2013,
    franchise: 'The Hobbit'
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6929'),
    title: 'The Hobbit: The Battle of the Five Armies',
    writer: 'J.R.R. Tolkien',
    year: 2012,
    franchise: 'The Hobbit',
    synopsis: 'Bilbo and Company are forced to engage in a war against an array of combatants and keep the Lonely Mountain from falling into the hands of a rising darkness.'
  }
]
24BCS0195> █

```

5. Retrieves all movies released in the 90s.

**Program:**

```
db.movies.find({year: {$gte: 1990,$lte: 1999}});
```

**Output:**

```

24BCS0195> db.movies.find({year: {$gte: 1990,$lte: 1999}});
[
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6924'),
    title: 'Fight Club',
    writer: 'Chuck Palahniuk',
    year: 1999,
    actors: [ 'Brad Pitt', 'Edward Norton' ]
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6925'),
    title: 'Pulp Fiction',
    writer: 'Quentin Tarantino',
    year: 1994,
    actors: [ 'John Travolta', 'Uma Thurman' ]
  }
]
24BCS0195> █

```

6. Retrieves all movies released before the year 2000 or after 2010.

### **Program:**

```
db.movies.find({$or: [{year: {$lt: 2000}}, {year: {$gt: 2010}}]});
```

### **Output:**

```

24BCS0195> db.movies.find({$or: [{year: {$lt: 2000}}, {year: {$gt: 2010}}]});
[
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6924'),
    title: 'Fight Club',
    writer: 'Chuck Palahniuk',
    year: 1999,
    actors: [ 'Brad Pitt', 'Edward Norton' ]
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6925'),
    title: 'Pulp Fiction',
    writer: 'Quentin Tarantino',
    year: 1994,
    actors: [ 'John Travolta', 'Uma Thurman' ]
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6927'),
    title: 'The Hobbit: An Unexpected Journey',
    writer: 'J.R.R. Tolkien',
    year: 2012,
    franchise: 'The Hobbit'
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6928'),
    title: 'The Hobbit: The Desolation of Smaug',
    writer: 'J.R.R. Tolkien',
    year: 2013,
    franchise: 'The Hobbit'
  },
  {
    _id: ObjectId('6a9f84ab43e672b0cffe6929'),
    title: 'The Hobbit: The Battle of the Five Armies',
    writer: 'J.R.R. Tolkien',
    year: 2012,
    franchise: 'The Hobbit',
    synopsis: 'Bilbo and Company are forced to engage in a war against an array of combatants and keep the Lonely Mountain from falling into the hands of a rising darkness.'
  }
]
24BCS0195> █

```

7. Insert synopsis to the title "The Hobbit: An Unexpected Journey" : "A reluctant hobbit, Bilbo Baggins, sets out to the Lonely Mountain with a spirited group of dwarves to reclaim their mountain home - and the gold within it - from the dragon Smaug."

### **Program:**

```
db.movies.updateOne({title: "The Hobbit: An Unexpected Journey"},{$set: {synopsis: "A reluctant hobbit, Bilbo Baggins, sets out to the Lonely Mountain with a spirited group of dwarves to reclaim their mountain home - and the gold within it - from the dragon Smaug."}});
```

### **Output:**

```
24BCS0195> db.movies.updateOne({title: "The Hobbit: An Unexpected Journey"},{$set: {synopsis: "A reluctant hobbit, Bilbo Baggins, sets out to the Lonely Mountain with a spirited group of dwarves to reclaim their mountain home - and the gold within it - from the dragon Smaug."}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
24BCS0195>
```

8. Insert a synopsis to the title "The Hobbit: The Desolation of Smaug" : "The dwarves, along with Bilbo Baggins and Gandalf the Grey, continue their quest to reclaim Erebor, their homeland, from Smaug. Bilbo Baggins is in possession of a mysterious and magical ring."

### **Program:**

```
db.movies.updateOne({title: "The Hobbit: The Desolation of Smaug"},{$set: {synopsis: "The dwarves, along with Bilbo Baggins and Gandalf the Grey, continue their quest to reclaim Erebor, their homeland, from Smaug. Bilbo Baggins is in possession of a mysterious and magical ring."}});
```

### **Output:**

```
24BCS0195> db.movies.updateOne({title: "The Hobbit: The Desolation of Smaug"},{$set: {synopsis: "The dwarves, along with Bilbo Baggins and Gandalf the Grey, continue their quest to reclaim Erebor, their homeland, from Smaug. Bilbo Baggins is in possession of a mysterious and magical ring."}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
24BCS0195>
```

9. Insert an actor named "Samuel L. Jackson" to the movie "Pulp Fiction"

### **Program:**

```
db.movies.updateOne({title: "Pulp Fiction"},{$push: {actors: "Samuel L. Jackson"}});
```

### **Output:**

```
24BCS0195> db.movies.updateOne({title: "Pulp Fiction"},{$push: {actors: "Samuel L. Jackson"}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
24BCS0195>
```

10. delete the movie "Pee Wee Herman's Big Adventure"

### **Program:**

```
db.movies.deleteOne({title: "Pee Wee Herman's Big Adventure"});
```

**Output:**

```
24BCS0195> db.movies.deleteOne({title: "Pee Wee Herman's Big Adventure"});  
{ acknowledged: true, deletedCount: 1 }  
24BCS0195> █
```

5. Insert the following documents into student collection.

```
([
  {
    "student_id": "STU001",
    "first_name": "Emma",
    "last_name": "Watson",
    "age": 20,
    "gpa": 3.9,
    "major": "Data Science",
    "enrolled": true,
    "skills": [
      "Python",
      "SQL",
      "Tableau"
    ],
    "address": {
      "city": "Boston",
      "state": "MA"
    }
  },
  {
    "student_id": "STU002",
    "first_name": "Liam",
    "last_name": "Neeson",
    "age": 22,
    "gpa": 3.1,
    "major": "Cybersecurity",
    "enrolled": true,
    "skills": [
      "Linux",
      "C++",
      "Wireshark"
    ],
    "address": {
      "city": "Austin",
      "state": "TX"
    }
  },
  {
    "student_id": "STU003",
    "first_name": "Sophia",
    "last_name": "Loren",
    "age": 21,
    "gpa": 3.6,
    "major": "Artificial Intelligence",
    "enrolled": false,
    "skills": [
      "Python",
      "PyTorch",
      "Git"
    ]
  }
])
```

```

        "address": {
            "city": "San Jose",
            "state": "CA"
        }
    },
    {
        "student_id": "STU004",
        "first_name": "Raj",
        "last_name": "Patel",
        "age": 23,
        "gpa": 3.4,
        "major": "Software Engineering",
        "enrolled": true,
        "skills": [
            "Java",
            "JavaScript",
            "AWS"
        ],
        "address": {
            "city": "Chicago",
            "state": "IL"
        }
    }
}
])

```

### **Program:**

```

db.student.insertMany([
    {student_id: "STU001",first_name: "Emma",last_name: "Watson",age: 20,gpa: 3.9,major: "Data
Science",enrolled: true,skills: [
        "Python",
        "SQL",
        "Tableau"
    ],address: {city: "Boston",state: "MA"
    }
    },
    {student_id: "STU002",first_name: "Liam",last_name: "Neeson",age: 22,gpa: 3.1,major:
"Cybersecurity",enrolled: true,skills: [
        "Linux",
        "C++",
        "Wireshark"
    ],address: {city: "Austin",state: "TX"
    }
    },
    {student_id: "STU003",first_name: "Sophia",last_name: "Loren",age: 21,gpa: 3.6,major:
"Artificial Intelligence",enrolled: false,skills: [
        "Python",
        "PyTorch",
        "Git"
    ],address: {city: "San Jose",state: "CA"
    }
    },
])

```

```
{student_id: "STU004",first_name: "Raj",last_name: "Patel",age: 23,gpa: 3.4,major: "Software Engineering",enrolled: true,skills: [
    "Java",
    "JavaScript",
    "AWS"
],address: {city: "Chicago",state: "IL"
}
}
]);
```

### **Output:**

```
24BCS0195> db.student.insertMany([
  {student_id: "STU001",first_name: "Emma",last_name: "Watson",age: 20,gpa: 3.9,major: "Data Science",enrolled: true,skills: ["Python","SQL","Tableau"],address: {city: "Boston",state: "MA"}},
  {student_id: "STU002",first_name: "Liam",last_name: "Neeson",age: 22,gpa: 3.1,major: "Cybersecurity",enrolled: true,skills: ["Linux","C++","Wireshark"],address: {city: "Austin",state: "TX"}},
  {student_id: "STU003",first_name: "Sophia",last_name: "Loren",age: 21,gpa: 3.6,major: "Artificial Intelligence",enrolled: false,skills: ["Python","PyTorch","Git"],address: {city: "San Jose",state: "CA"}},
  {student_id: "STU004",first_name: "Raj",last_name: "Patel",age: 23,gpa: 3.4,major: "Software Engineering",enrolled: true,skills: ["Java","JavaScript","AWS"],address: {city: "Chicago",state: "IL"}}]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a9f888943e672b0cffe692c'),
    '1': ObjectId('6a9f888943e672b0cffe692d'),
    '2': ObjectId('6a9f888943e672b0cffe692e'),
    '3': ObjectId('6a9f888943e672b0cffe692f')
  }
}
24BCS0195>
```

Find the output for the following queries

1. Find the number of students with a GPA higher than 3.5

### **Program:**

```
db.student.countDocuments({gpa: {$gt: 3.5}});
```

### **Output:**

```
24BCS0195> db.student.countDocuments({gpa: {$gt: 3.5}});
2
24BCS0195>
```

2. Find all the students who know both Python and PyTorch

### **Program:**

```
db.student.find({skills: {$all: ["Python","PyTorch"]}});
```

### **Output:**

```
24BCS0195> db.student.find({skills: {$all: ["Python","PyTorch"]}});
[
  {
    _id: ObjectId('6a9f888943e672b0cffe692e'),
    student_id: 'STU003',
    first_name: 'Sophia',
    last_name: 'Loren',
    age: 21,
    gpa: 3.6,
    major: 'Artificial Intelligence',
    enrolled: false,
    skills: [ 'Python', 'PyTorch', 'Git' ],
    address: { city: 'San Jose', state: 'CA' }
  }
]
24BCS0195> █
```

3. Find the students where the first skill in their list is Python

**Program:**

```
db.student.find({"skills.0": "Python"});
```

**Output:**



```

24BCS0195> db.student.find({skills: {$all: ["Python","PyTorch"]}});
[
  {
    _id: ObjectId('6a9f888943e672b0cffe692e'),
    student_id: 'STU003',
    first_name: 'Sophia',
    last_name: 'Loren',
    age: 21,
    gpa: 3.6,
    major: 'Artificial Intelligence',
    enrolled: false,
    skills: [ 'Python', 'PyTorch', 'Git' ],
    address: { city: 'San Jose', state: 'CA' }
  }
]
24BCS0195> db.student.find({"skills.0": "Python"});
[
  {
    _id: ObjectId('6a9f888943e672b0cffe692c'),
    student_id: 'STU001',
    first_name: 'Emma',
    last_name: 'Watson',
    age: 20,
    gpa: 3.9,
    major: 'Data Science',
    enrolled: true,
    skills: [ 'Python', 'SQL', 'Tableau' ],
    address: { city: 'Boston', state: 'MA' }
  },
  {
    _id: ObjectId('6a9f888943e672b0cffe692e'),
    student_id: 'STU003',
    first_name: 'Sophia',
    last_name: 'Loren',
    age: 21,
    gpa: 3.6,
    major: 'Artificial Intelligence',
    enrolled: false,
    skills: [ 'Python', 'PyTorch', 'Git' ],
    address: { city: 'San Jose', state: 'CA' }
  }
]
24BCS0195>

```

4. Update all students majoring in "Data Science" and adds "MongoDB" to their skills array.

**Program:**

```
db.student.updateMany({major: "Data Science"},{$addToSet: {skills: "MongoDB"}});
```

**Output:**

```
24BCS0195> db.student.updateMany({major: "Data Science"},{$addToSet: {skills: "MongoDB"}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
24BCS0195>
```

5. To add a new skill to a Student\_id :”STU001”

**Program:**

```
db.student.updateOne({student_id: "STU001"},{$addToSet: {skills: "Docker"}});
```

**Output:**

```
24BCS0195> db.student.updateOne({student_id: "STU001"},{$addToSet: {skills: "Docker"}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
24BCS0195> █
```

6. Delete all the documents in student collection.

**Program:**

```
db.student.deleteMany({});
```

**Output:**

```
24BCS0195> db.student.deleteMany({});
{ acknowledged: true, deletedCount: 4 }
24BCS0195> █
```